A binary number is a combination of 1s and 0s. Its $n^{th}$ least significant digit is the $n^{th}$ digit starting from the right starting with 1. Given a decimal number, convert it to binary and determine the value of the the $4^{th}$ least significant digit.

**Example**

number = 23

- Convert the decimal number 23 to binary number: $23^{10} = 2^4 + 2^2 + 2^1 + 2^0 = (10111)_2$.
- The value of the $4^{th}$ index from the right in the binary representation is 0.

**Function Description**

Complete the function fourthBit in the editor below.

fourthBit has the following parameter(s):

int number: a decimal integer

- Convert the decimal number 23 to binary number: $23^{10} = 2^4 + 2^2 + 2^1 + 2^0 = (10111)_2$.
- The value of the $4^{th}$ index from the right in the binary representation is 0.

**Function Description**

Complete the function fourthBit in the editor below.

fourthBit has the following parameter(s):

int number: a decimal integer

Returns:

int: an integer 0 or 1 matching the 4th least significant digit in the binary representation of number.

**Constraints**

$0 \leq number < 2^{31}$

**Input Format for Custom Testing**

Input from stdin will be processed as follows and passed to the function.

The only line contains an integer, number.

**Sample Case 0**

**Sample Input 0**

STDIN  Function

-----  --------

32  → number = 32

**Sample Output 0**

0

**Explanation 0**

· Convert the decimal number 32 to binary number: $32_{10} = (100000)_2$.

· The value of the 4th index from the right in the binary representation is 0.

**Sample Case 1**

**Sample Input 1**

STDIN Function

----- --------

77 → number = 77

**Sample Output 1**

1

**Explanation 1**

· Convert the decimal number 77 to binary number: $77_{10} = (1001101)_2$.

· The value of the 4th index from the right in the binary representation is 1.

**Answer:** (penalty regime: 0 %)

Reset answer

```c
/*
 * Complete the 'fourthBi
 *
 * The function is expect
 * The function accepts I
 */

int fourthBit(int number)
{
    int binary[32];
    int i=0;
    while(number>0)
    {
        binary[i]=numbe
        number/=2;
        i++;
    }
    if(i>=4)
    {
        return binary[3
    }
    else
    return 0;
}
```

| | Test |
|---|---|
| ✓ | printf("%d", fourthBit(32)) |
| ✓ | printf("%d", fourthBit(77)) |

| | Test |
|---|---|
| ✓ | `printf("%d", fourthBit(32))` |
| ✓ | `printf("%d", fourthBit(77))` |

Passed all tests! ✓

Question **2**

Correct

Marked out of 1.00

Flag question

Determine the factors of a number (i.e., all positive integer values that evenly divide into a number) and then return the $p^{th}$ element of the list, sorted ascending. If there is no $p^{th}$ element, return 0.

**Example**

n = 20

p = 3

Determine the factors of a number (i.e., all positive integer values that evenly divide into a number) and then return the $p^{th}$ element of the list, sorted ascending. If there is no $p^{th}$ element, return 0.

**Example**

n = 20

p = 3

The factors of 20 in ascending order are {1, 2, 4, 5, 10, 20}. Using 1-based indexing, if p = 3, then 4 is returned. If p > 6, 0 would be returned.

**Function Description**

Complete the function pthFactor in the editor below.

pthFactor has the following parameter(s):

int n: the integer whose factors are to be found

pthFactor has the following parameter(s):

int n: the integer whose factors are to be found

int p: the index of the factor to be returned

Returns:

int: the long integer value of the $p^{th}$ integer factor of n or, if there is no factor at that index, then 0 is returned

**Constraints**

$1 \leq n \leq 10^{15}$

$1 \leq p \leq 10^{9}$

Input Format for Custom Testing

Input from stdin will be processed as follows and passed to the function.

The first line contains an integer n, the number to factor.

The second line contains an integer p, the 1-based index of the factor to return.

**Sample Case 0**

**Sample Input 0**

STDIN Function

----- --------

10 → n = 10

3 → p = 3

**Sample Output 0**

5

**Explanation 0**

Factoring n = 10 results in {1, 2, 5, 10}. Return the p = 3rd factor, 5, as the answer.

**Sample Case 1**

**Sample Input 1**

STDIN Function

----- --------

10 → n = 10

5 → p = 5

**Sample Output 1**

0

**Explanation 1**

Factoring n = 10 results in {1, 2, 5, 10}. There are only 4 factors and p = 5, therefore 0 is returned as the answer.

**Sample Case 2**

**Sample Input 2**

STDIN Function

----- --------

1 → n = 1

1 → p = 1

**Sample Output 2**

1

**Explanation 2**

Factoring n = 1 results in {1}. The p = 1st factor of 1 is returned as the answer.

**Answer:** (penalty regime: 0 %)

Reset answer

```
/*
 * Complete the 'pthFacto
 *
 * The function is expect
 * The function accepts f
 *  1. LONG_INTEGER n
 *  2. LONG_INTEGER p
 */

long pthFactor(long n, lo
{
    int count=0;
    for(long i=1;i<=n;++i
    {
        if(n%i==0)
        {
            count++;
            if(count==p)
```

Reset answer

```
/*
 * Complete the 'pthFacto
 *
 * The function is expect
 * The function accepts f
 *  1. LONG_INTEGER n
 *  2. LONG_INTEGER p
 */

long pthFactor(long n, lo
{
    int count=0;
    for(long i=1;i<=n;++i
    {
        if(n%i==0)
        {
            count++;
            if(count==p)
            {
                return i;
            }
        }
    }
    return 0;
}
```

| | Test |
|---|---|
| ✓ | printf("%ld", pthFactor(10 |

```
22          }
23      }
24      return 0;
25  }
```

| | Test |
|---|---|
| ✓ | printf("%ld", pthFactor(10, |
| ✓ | printf("%ld", pthFactor(10, |
| ✓ | printf("%ld", pthFactor(1, |

Passed all tests! ✓

Finish review

Quiz navigation

1 2

Show one page at a time

Finish review

You are a bank account hacker. Initially you have 1 rupee in your account, and you want exactly ***N*** rupees in your account. You wrote two hacks, first hack can multiply the amount of money you own by 10, while the second can multiply it by 20. These hacks can be used any number of time. Can you achieve the desired amount ***N*** using these hacks.

**Constraints:**

1<=T<=100

1<=N<=10^12

**Input**

· The test case contains a single integer N.

**Output**

For each test case, print a single line containing the string "1" if you can make exactly N rupees or "0" otherwise.

SAMPLE INPUT

1

SAMPLE OUTPUT

1

SAMPLE INPUT

2

SAMPLE OUTPUT

0

**Answer:** (penalty regime: 0 %)

Reset answer

```
/*
 * Complete the 'myFunc'
 *
 * The function is expect
 * The function accepts I
 */
```

```c
/*
 * Complete the 'myFunc'
 *
 * The function is expect
 * The function accepts I
 */

int myFunc(int n)
{
    return n==1||n%10==0;
}
```

| | Test |
|---|---|
| ✓ | printf("%d", myFunc(1)) |
| ✓ | printf("%d", myFunc(2)) |
| ✓ | printf("%d", myFunc(10)) |
| ✓ | printf("%d", myFunc(25)) |
| ✓ | printf("%d", myFunc(200)) |

| | Test |
|---|---|
| ✓ | `printf("%d", myFunc(1))` |
| ✓ | `printf("%d", myFunc(2))` |
| ✓ | `printf("%d", myFunc(10))` |
| ✓ | `printf("%d", myFunc(25))` |
| ✓ | `printf("%d", myFunc(200))` |

Passed all tests! ✓

Question **2**

Correct

Marked out of 1.00

Flag question

Find the number of ways that a given integer, ***X***, can be expressed as the sum of the $N^{th}$ powers of unique, natural numbers.

For example, if ***X = 13*** and ***N = 2***, we have to find all combinations of unique squares adding up to ***13***. The only

**Function Description**

Complete the powerSum function in the editor below. It should return an integer that represents the number of possible combinations.

powerSum has the following parameter(s):

X: the integer to sum to

N: the integer power to raise numbers to

Input Format

The first line contains an integer ***X***.

The second line contains an integer ***N***.

**Constraints**

***1 ≤ X ≤ 1000***

***2 ≤ N ≤ 10***

**Output Format**

Output a single integer, the number of possible combinations calculated.

**Sample Input 0**

10

2

**Sample Output 0**

1

**Explanation 0**

If ***X = 10*** and ***N = 2***, we need to find the number of ways that ***10*** can be represented as the sum of squares of unique numbers.

$$10 = 1^2 + 3^2$$

This is the only way in which ***10*** can be expressed as the sum of unique squares.

**Sample Input 1**

100

2

**Sample Output 1**

3

**Explanation 1**

$100 = (10^2) = (6^2 + 8^2) = (1^2 + 3^2 + 4^2 + 5^2 + 7^2)$

**Sample Input 2**

100

3

Sample Output 2

1

**Explanation 2**

***100*** can be expressed as the sum of the cubes of ***1, 2, 3, 4***.

***(1 + 8 + 27 + 64 = 100)***. There is no other way to express ***100*** as the sum of cubes.

**Answer:** (penalty regime: 0 %)

Reset answer

```
1  /*
2   * Complete the 'powerSum
3   *
4   * The function is expect
5   * The function accepts f
6   *  1. INTEGER x
7   *  2. INTEGER n
8   */
9  #include<math.h>
10
11 int powerSum(int x, int m
12 {
13     int p=pow(m,n);
14     if(p==x)
15     {
16         return 1;
17     }
18     if(p>x)
19     {
20         return 0;
21     }
22     return powerSum(x-p,m
```

```
 *   2. INTEGER n
 */
#include<math.h>

int powerSum(int x, int m
{
    int p=pow(m,n);
    if(p==x)
    {
        return 1;
    }
    if(p>x)
    {
        return 0;
    }
    return powerSum(x-p,m
}
```

| | Test |
|---|---|
| ✓ | `printf("%d", powerSum(10, 1` |

Passed all tests! ✓

Finish review